Codeer Agent Skill 客戶安裝指南

這份指南協助你完成 Codeer CLI 與 Codeer Agent Skill 安裝。

重點不是避開 Web UI，而是讓 Codex / Claude Code 能參與完整的 Codeer Agent 生命週期：從資料整理、Agent 規劃、Behavior Cases、Production History 分析，到下一輪迭代。

先分清楚：CLI 和 Skill 各自負責什麼？

Codeer CLI

Codeer CLI 是安裝在本機的 codeer 指令工具。

它負責：

- 登入 Codeer
- 連接 Codeer API
- 執行 codeer agent、codeer eval、codeer history、codeer kb 等指令
- 驗證 API key、workspace、agent 權限是否正常
- 使用 codeer skill 安裝、列出或移除 Codeer Skills（CLI 0.1.17 起）

Codeer Agent Skill

Codeer Agent Skill 是安裝到 Codex / Claude Code 的工作說明。

它負責讓 assistant 知道：

- Codeer Agent 生命週期怎麼走
- 如何從 KB / 檔案討論並建立 Agent v0
- 如何產生 Behavior Cases
- 如何分析 Production History / Negative Feedback
- 什麼時候要先讓你看會改哪些內容，再由你確認是否套用或發布

Codeer Agent Skill 可以發揮什麼威力

1. 從 KB / 檔案開始規劃並創建 Agent v0

把文件、FAQ、SOP、產品資料或資料夾交給 Codex / Claude Code，先討論Agent 範圍、知識邊界與初版設計。

2. 產生 Behavior Cases

依照目標使用情境建立行為分類、測試案例與評分標準，讓品質討論有具體依據。

3. 根據失敗問題作出調整

失敗不一定代表 prompt 錯；也可能是 KB、Agent、retrieval、Standard / Rubric 需要修正。

4. 分析 Production History / Negative Feedback

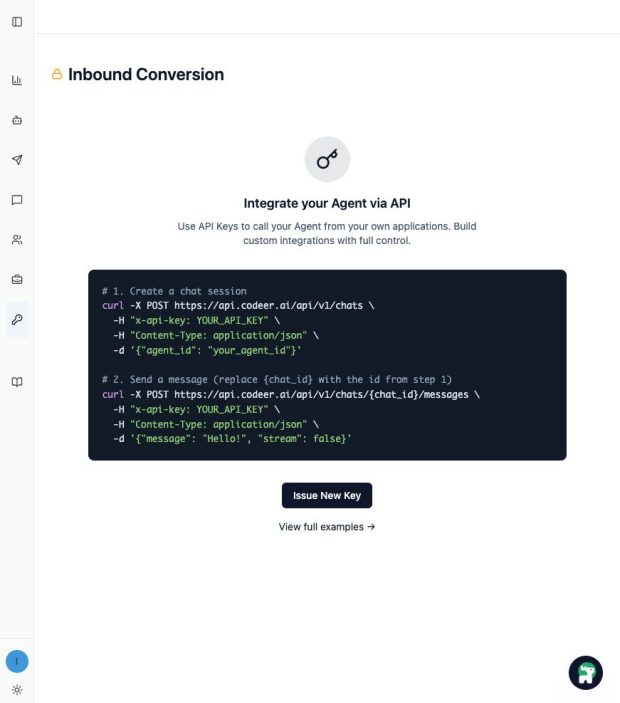
從真實對話和負評中找出模式、缺口與高風險場景，讓下一輪改善有證據。

5. 協助你開始下一輪的迭代

先整理問題、提出修正、預覽會改哪些內容，再由你確認是否套用或發布。

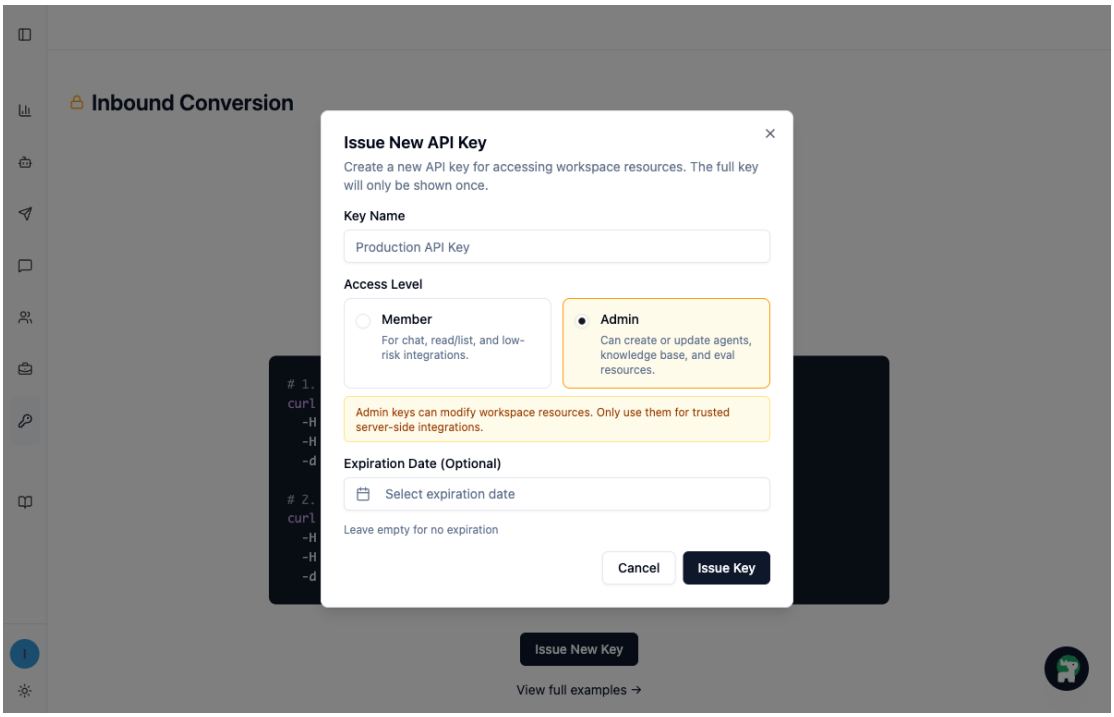
Step 1：在 Codeer 建立 API Key

請使用目標 workspace 的 Admin API key。Skill 會透過 CLI 做 Agent、KB、Behavior Cases 和 History 相關工作，因此初次設定建議使用 Admin權限。



Codeer API Keys page

在 workspace 頁面左側打開 API Keys，點選 Issue New Key。



Issue New API Key dialog

輸入容易辨識的名稱，例如 Codex 或 Claude Code，Access level 選Admin。

完整 API key 只會顯示一次。請立即存到安全位置，不要貼進聊天、prompt、Git repo、專案 .env 或 .claude/settings.json。

Step 2：安裝 Codeer CLI

Codeer Agent Skill 會透過本機 codeer CLI 操作 Codeer API。建議使用pipx 安裝，讓 CLI 跟專案 Python 環境分開，比較容易維護。

安裝 CLI：

```
pipx install codeer-cli
```

確認指令可用：

```
codeer --help
```

如果沒有 pipx：

```
python -m pip install --user pipx
python -m pipx ensurepath
```

重開 terminal 後，再執行：

```
pipx install codeer-cli
```

維護更新：

```
pipx upgrade codeer-cli
codeer check
```

不建議把 python -m pip install codeer-cli 當作主要安裝方式。它可以作為fallback，但 pipx 較不容易受到專案套件版本影響。

Step 3：設定本機 CLI Profile

Profile 會把 API key 存在使用者層級的 Codeer CLI config，不需要放進Codex 或 Claude Code 的設定檔。

下面的 work 只是範例 profile 名稱。你可以換成自己容易辨識的名字，例如codeer、prod、client-a 或 support-agent。如果你改了名稱，後面的codeer profile use ... 也要使用同一個名字。

新增 profile：

```
codeer profile add work
```

CLI 會請你貼上 API key，輸入時不會顯示在畫面上。

選用 profile：

```
codeer profile use work
```

檢查 CLI 與 API 是否可用：

```
codeer check
codeer agent list
```

這只能確認 CLI 和 API credential 正常，還不是 Skill 驗證。

如果專案只有一個預設 Agent，可以選填：

```
export CODEER_AGENT_ID=<agent-id>
```

不要把 CODEER_API_KEY 放進 .claude/settings.json、專案 .env、聊天訊息或 Git commit。

Step 4：安裝 Codeer Agent Skill

使用 Codeer CLI 0.1.17 或更新版本安裝。安裝 Skill 不需要 Codeer API key；操作 Codeer 伺服器時才需要前面設定的 profile。

Claude Code

```
codeer skill install codeer-agent --target claude
```

Codex

```
codeer skill install codeer-agent --target codex
```

預設安裝到使用者層級：Codex 使用 ~/.agents/skills/codeer-agent，Claude Code 使用 ~/.claude/skills/codeer-agent。加上 --scope project會改成目前目錄下的 .agents/skills/ 或 .claude/skills/。

使用 codeer skill list 查看安裝狀態。--dry-run 會下載並驗證來源、顯示安裝目的地，但不寫入安裝檔案；--ref <tag-or-commit> 可固定來源版本，省略時使用 main。來源版本必須包含 skills/codeer-agent 資料夾。

如果安裝後沒有看到 Skill，請重新啟動 Codex；Claude Code 可執行/reload-skills。

也可以使用 Codex 內建的 installer：

```
$skill-installer install https://github.com/codeer-ai/codeer-skills/tree/main/skills/codeer-agent
```

這種安裝由原本的 installer 管理，codeer skill 不會接管或移除。

更新與移除 Skill

再次執行 install 就會更新未修改的 CLI 管理套件，不必先移除。先用 --dry-run 預覽，再安裝 main 的版本；也可用 --ref 選擇指定版本。Claude Code 請將 --target codex 換成--target claude；若原本使用 --scope project，以下指令也要加上同一選項。

```
codeer skill install codeer-agent --target codex --dry-run
codeer skill install codeer-agent --target codex
codeer skill install codeer-agent --target codex --ref <tag-or-commit>
```

相同內容會回傳 already_installed，不改動檔案；更新成功會回傳 updated。預覽會回傳 would_install、would_update 或 already_installed。更新會先下載並驗證完整新版，再替換舊目錄並清除過期檔案；替換失敗時還原舊版。若還原也失敗，錯誤訊息會提供保留備份的位置，供手動復原。

內容已修改、手動安裝或透過符號連結管理的目錄會被保留；請先保存自訂內容，再手動處理。若要移除未修改的 CLI 管理套件：

```
codeer skill uninstall codeer-agent --target codex --dry-run
codeer skill uninstall codeer-agent --target codex
```

移除 CLI 不會一併移除 Skill。

Step 5：驗證 Skill 是否被載入

請在 Codex / Claude Code 中問下面這句。這是在確認 assistant 是否真的知道Codeer Agent 生命週期，而不是只確認 CLI 能不能連線。

> Please confirm whether the Codeer Agent Skill is loaded. Explain how you can help me plan an Agent from KB/files, create behavior cases, analyze production history, and show me proposed changes before applying them.

理想回答應該提到：

- Agent lifecycle
- Behavior Cases
- Production History
- 在建立、更新或發布 Codeer 資源前，會先向你確認

Codeer Agent Skill 會怎麼和你協作？

Skill 的角色是把 Agent 改善流程拆成可以討論、可以確認的步驟。它不只是幫你下指令，也會協助判斷下一步該改什麼。

- 先了解你的目標、資料來源和目前 Agent 狀態，再提出適合的工作順序。
- 需要建立或調整 Agent、KB、Behavior Cases、Rubrics 時，會先說明它準備做什麼。
- 真正套用修改前，會先讓你預覽會改哪些內容，或用修改前後差異的方式讓你確認。
- 發布新版 Agent 前，會先確認測試結果、失敗分析和修改內容都已經被看過。

CLI 常見問題快速排除

codeer command not found

執行：

```
pipx ensurepath
```

然後重開 terminal 再試一次。

codeer check 顯示 401 / 403

API key 可能過期、被撤銷或權限不足。請重新建立 Admin workspace key 並更新profile。

```
codeer profile add work
codeer profile use work
codeer check
```

看到錯的 workspace

目前 profile 使用了另一個 workspace 的 API key。改用正確 profile 後再執行：

```
codeer check
```

找不到 Agent

確認 CODEER_AGENT_ID 是否屬於同一個 workspace；不確定時先 unset 後列出workspace agents：

```
unset CODEER_AGENT_ID
codeer agent list
```

安裝完成後，下一步做什麼？

如果你還沒有 Agent

可以問：

> 這些是我的 KB / 文件資料夾，請先跟我討論適合建立什麼 Agent v0、需要哪些資料、有哪些風險邊界。

如果你有 Agent，但還沒有 Behavior Cases

可以問：

> 請根據這個 Agent 的任務，設計 behavior categories、cases 和 rubrics，先讓我審核再套用。

如果你有 Behavior Cases，但有些沒有通過

可以問：

> 請分析失敗原因，判斷應該修 KB、Agent instructions、retrieval，還是 Standard / Rubric，並先讓我看會改哪些內容。

如果你的 Agent 已經上線

可以問：

> 請分析 Production History / Negative Feedback，找出常見失敗模式、缺少的測試覆蓋，以及下一輪應優先處理的問題。

如果你準備開始下一輪迭代

可以問：

> 請根據目前問題、測試結果與歷史紀錄，提出下一版修改計畫；先讓我看證據與會改哪些內容，再決定是否套用或發布。